If the bet receives a `MarketConfirmed` event, it persists the event and keeps in memory the information in the `OpenState` by setting `marketConfirmed` to `Some(true)`; otherwise, it goes to the `FailedState`.

Let's look at the second validation when opening the bet: `requestFundsReservation`. To check whether the wallet can reserve the necessary funds, there is no business requirement like in the market. In general, it is cleaner to `tell` than to `ask`. The wallet may also have to make calls to other services: for example, to check that the customer is not overspending or breaking a law. If you expect a chain of calls, `tell` is usually the best option. Here's the code:

```
object Bet {
  ...

  private def requestFundsReservation(
      command: Open,
      sharding: ClusterSharding,
      context: ActorContext[Command]): Unit = {
    val walletRef =
      sharding.entityRefFor(Wallet.typeKey, command.walletId)       ❶
    val walletResponseMapper: ActorRef[Wallet.UpdatedResponse] =    ❷
      context.messageAdapter(rsp => RequestWalletFunds(rsp))

    walletRef ! Wallet.ReserveFunds(                                ❸
      command.stake,
      walletResponseMapper)
  }
  ...
}
```

❶ Retrieves the wallet's ActorRef

❷ Creates a method to adapt the wallet's response to the Bet's protocol

❸ Tells the wallet to reserve funds